Chapter 2: The Basics of Measurement

2.1 The Representational Theory of Measurement

What is Measurement?

Measurement is mapping real-world attributes of entities (like height, temperature, or software complexity) to numbers or symbols in a way that reflects our intuitive understanding.

Why is it important?

In physical sciences (like measuring length or temperature), we have mature measurement tools and theories. In software, we often lack such maturity and tools, so building up a **theory of measurement** is necessary.

Representational Theory of Measurement

This theory helps ensure that when we assign numbers to attributes, we preserve the relationships we see in the real world.

Example (Height):

- We compare people (Frankie, Wonderman, Peter) and see who is taller (empirical relation).
- When we measure with a ruler (mapping to numbers), it should preserve "taller than" relation (representation condition).

Key Example (Software):

- Suppose we have 4 contact management programs (A, B, C, D).
- 100 users rank them based on **functionality**.
- If most users say "C is better than A," then our measuring system must assign a higher number to C than A.

Important:

- **Empirical relations**: How people/things/entities relate in the real world (like taller than, more complex, faster).
- **Mapping**: Assigning numbers/symbols.
- **Representation Condition**: The real-world relationship must match the numeric relationship in your measurement.

2.2 Measurement and Models

What is a Model?

A model is an abstraction—a simplified way to represent reality. Models help us focus on important aspects.

Example:

When measuring code length (LOC), you must model what counts as a "line" (do you include comments? blank lines?).

Direct vs Derived Measurement

- **Direct measurement**: Assign a value directly (e.g., number of lines of code, number of hours worked).
- **Derived measurement**: Combine measurements (e.g., productivity = LOC / person-month).

Why Models Matter:

Models guide what you measure, how you measure it, and how you interpret the results.

Table Example:

Entity	Attribute	Measure
Program code	Length	Number of LOC
Testing process	Duration	Hours from start to finish

Program code	Reliability	Mean time to failure

2.3 Measurement Scales and Scale Types

Different Scale Types

Knowledge of scale types is essential – they determine how you can analyze and compare data.

1. Nominal

- Classification only (no order!).
- Example: Categorizing bugs by type (syntax, logic, runtime).
- You can count how many in each group but not compare magnitudes.

2. Ordinal

- Ordered categories.
- Example: Ranking software as "low," "medium," or "high" complexity.
- You can say which is more/less but not how much more.

3. Interval

- Ordered, with equal intervals, but no true zero point.
- Example: Temperature in Celsius or Fahrenheit. Differences are meaningful, but ratios are not.

4. Ratio

- Like interval, but with a meaningful zero. Can say "twice as much."
- Example: Program length in LOC, durations, cost.

5. Absolute

- Simple counting, the most restrictive scale.
- Example: Number of defects found during testing. Only one way to count.

Summary Table:

Scale	What's allowed	Example
Nominal	Categories	Bug types (syntax, logic)
Ordinal	Order, no magnitudes	Low/Medium/High complexity
Interval	Order & equal intervals	Celsius temperature
Ratio	Order, intervals, true zero	LOC, duration, effort
Absolute	Unique counting	Number of test failures

2.4 Meaningfulness in Measurement

Understanding whether statements/calculations are **meaningful** is critical:

- You can only use certain mathematical operations for certain scales.
  - You can't average "nominal" data in a meaningful way.
  - You can only take meaningful ratios if your scale is ratio or absolute.

Example:

- "Fred is twice as tall as Jane" is meaningful (ratio scale).
- "Tokyo is twice as hot as London" (in Celsius): not meaningful, since Celsius has no true zero.
- "Average software complexity score" (ordinal) is **not** meaningful if calculated as a mean, but median or mode is.

Applying Measures – Statistical Methods

For each scale:

Scale	Meaningful Statistics
Nominal	Mode, frequency counts
Ordinal	Median, percentiles, mode
Interval	Mean, standard deviation
Ratio	All arithmetic (mean, ratios)

Absolute	All arithmetic

Subjective vs. Objective Measures

- **Objective**: Anyone measuring gets the same answer (e.g., LOC).
- **Subjective**: Depends on the individual or situation (e.g., rating usability 1–5).

Combining Multiple Measures

Sometimes, you need to assess an attribute (like quality or complexity) via multiple sub-attributes.

- Can't always combine into a single number unless it's mathematically defensible.
- Sometimes ranking or n-tuples (vectors) are more appropriate.

Answers to End-of-Chapter Questions

I'll answer each question thoroughly as requested for exam prep.

1. Four Questions at the Start

a. How much must we know about an attribute before it is reasonable to consider measuring it?

You need enough understanding of the attribute to define empirical relations (e.g., what does "complex" mean in software?). If you cannot consistently distinguish levels or categories (e.g., more complex, more reliable), then meaningful measurement isn't possible.

b. How do we know if we have really measured the attribute we wanted to measure?

If the numeric assignment (measure) preserves the empirical relations we intuitively understand, then we have a valid measurement (representation condition). For example, if we measure "complexity" by LOC, does a higher LOC

always mean "more complex"? If not, the measure may not align well with the attribute.

c. Using measurement, what meaningful statements can we make about an attribute and the entities that possess it?

Only statements allowed by the measurement's scale type. With ratio scales, we can state things like "Module A is twice as long as Module B." With ordinal scales, only statements about order (e.g., "A is more complex than B") are meaningful.

d. What meaningful operations can we perform on measures?

That depends on the scale:

- Nominal: Count, mode.
- Ordinal: Median, percentiles.
- Interval: Mean, standard deviation (no ratios).
- Ratio/Absolute: Mean, ratio, all algebraic operations.

---

2. Scale Types and Measuring Complexity

a. List, in increasing order of sophistication, the five most important measurement scale types.

1. Nominal
2. Ordinal
3. Interval
4. Ratio
5. Absolute

b. If complexity is ranked 1 (trivial) to 5 (incomprehensible), what is the scale type? How do you know? How could you meaningfully measure the average of a set of modules?

- Scale type: **Ordinal**, because the numbers represent order, not equal intervals or absolute values.
- You can calculate the **median** (the middle value when modules are ordered by complexity).

- Calculating the mean is **not** meaningful for ordinal data.

---

3. Examples of Common Scale Types in Software

Nominal:

- Entity: Software bug
- Attribute: Bug type (syntax, logic, runtime)
- Product/process/resource: Product

Ratio:

- Entity: Program
- Attribute: Number of lines of code (LOC)
- Product/process/resource: Product

---

4. Define Measurement and Representation Condition

Measurement:

The mapping from real-world entities' attributes to numbers or symbols in such a way that the relationships between the entities (empirical relations) are preserved in the numeric assignments.

Representation Condition:

A measure must preserve and reflect the relationships that exist in the empirical world in the numeric world; if X is “more” (e.g., taller/more complex) than Y in the real world, then M(X) > M(Y) in numbers.

---

5. Example 2.5 – Numerical Assignments Representation Condition

Representation condition: If one is taller than another, the numerical value must be greater.

Let’s recall:

- Frankie > Wonderman > Peter

a. M(Wonderman) = 100; M(Frankie) = 90; M(Peter) = 60

Invalid: Wonderman should not be less than Frankie.

b. M(Wonderman) = 100; M(Frankie) = 120; M(Peter) = 60

Valid: Frankie > Wonderman > Peter numerically.

c. M(Wonderman) = 100; M(Frankie) = 120; M(Peter) = 50

Valid: Same as before.

d. M(Wonderman) = 68; M(Frankie) = 75; M(Peter) = 40

Valid: Same order preserved.

---

6. Example 2.6 – Empirical Relations Mapping

Check which mappings reflect Data-loss > Incorrect output > Delayed Response.

a. 6, 6, 69 (delayed, incorrect, data-loss):

Invalid. Delayed and incorrect are indistinguishable.

b. 1, 2, 3

Valid. 3 > 2 > 1 matches criticality ordering.

c. 6, 3, 2

Invalid. Data-loss (2) < Delayed (6), which is opposite.

d. 0, 1, 0.5

Valid. 1 (incorrect output), 0.5 (data-loss), 0 (delayed). Doesn't match ordering unless system says higher is less critical, but if highest number = most critical, then this does not match.

In standard interpretations, **b** is correct.

---

7. Two Measures of Criticality

Suppose:

- Syntactic < Semantic < System Crash (Crash most critical).

Measure 1:

M(syntactic) = 1

M(semantic) = 2

M(system crash) = 3

Measure 2:

M(syntactic) = 10

M(semantic) = 20

M(system crash) = 100

Both are ordinal scales. They're related by a monotonic increasing function.

---

8. Faults per KLOC & Quality

"Twice as many faults per KLOC does not mean half as good quality."

Because fault detection is not a direct, ratio-scale indicator of quality; it could reflect many factors (code complexity, test quality, etc.).

---

9. LOC as a Measure

Lines of code (LOC) is not a "bad" measure in itself — it's a valid measure of **size**. Whether it's a good measure for other attributes (like complexity or productivity) depends on context.

---

10. M4, M5 in Example 2.16

M4 and M5 assign the same number (1) to both trivial and simple categories. This means the mapping does not preserve order (ordinal scale violated).

---

11. Affine Transformations in Example 2.18

Let's set up mappings:

- M1: 1, 2, 3, 4, 5
- M2: 0, 2, 4, 6, 8
- M3: 3.1, 5.1, 7.1, 9.1, 11.1

Affine transformation is of the form Mnew = a * Mold + b

a. M1 to M2:

0 = a*1 + b → b = -a*
2 = a
2 + b → b = 2 - 2a
Set equal: -a = 2 - 2a ⟹ a = 2, b = -2
So: M2 = 2*M1 - 2

b. M2 to M1:

M1 = (M2 + 2)/2

c. M2 to M3:

Try for interval step: M3 = a*M2 + b

0 → 3.1; 2 → 5.1; 4 → 7.1

So a = 1, b = 3.1 for 0; 2 → 5.1 = a*2 + 3.1 → a = 1

d. M3 to M2:

M2 = M3 - 3.1

e. M1 to M3:

M3 = 2*M1 + 1.1

---

12. Duration of Process as Ratio Scale

Time duration can be measured in hours, days, weeks – all ratio scales, as zero means "no duration," and ratios make sense.

Transformations: days = 24 * hours.

---

13. Meaningful Statements

a. The length of Program A is 50.

Not meaningful (we don't know the unit/scale).

b. The length of Program A is 50 executable statements.

Meaningful (absolute number).

c. Program A took 3 months to write.

Meaningful (duration).

d. Program A is twice as long as Program B.

Meaningful (ratio).

e. Program A is 50 lines longer than Program B.

Meaningful (interval and ratio).

f. The cost of maintaining program A is twice that of maintaining B.

Meaningful if cost is measured on a ratio scale.

g. Program B is twice as maintainable as A.

Not meaningful unless maintainability is on a ratio scale.

h. Program A is more complex than B.

Meaningful as an ordinal statement.

---

14. Meaningfulness (Definition + Examples)

Meaningfulness means:

A statement's truth is unchanged by any allowed transformation of the scale (e.g., from meters to feet).

- "The average size of Windows apps is four times that of Linux":

  Meaningful if size is measured on a ratio scale.

- "Tool A achieved higher average usability than B":

  If usability ratings are ordinal, comparing averages is not meaningful; comparing median or mode is.

---

15. Mean with Interval Data

For interval scale, affine transformations are allowed (additive and multiplicative).

Mean is preserved under such transformations (e.g., mean temperature in Celsius and Fahrenheit).

---

16. Mode vs Median for Nominal

For nominal data, no order exists—so median (middle value) is not meaningful, but **mode** (most common value) is.

---

17. Average Complexity (Ordinal Data)

a. Use **median** to report central tendency.

b. Mean is not meaningful since order alone is preserved—not equal intervals.

c. Criteria for assigning complexity:

- Number of decision points (if-else, case, etc.)
- Level of coupling with other modules

Assumptions: All raters use the same rubric.

---

18. Multiple Attributes for Quality

- Draw a partial order diagram (like Figure 2.13).
- Not possible to map to real numbers preserving all relations due to incomparabilities (e.g., "train" is cheaper but slower).
- Use a vector (e.g., (performance, cost)) for mapping.

---

19. Rescaling Programmer Productivity

P = L/E

Any rescaling of L or E produces: P′ = (αL) / (βE) = (α/β)P

So every rescaling is of the form P′ = k*P

---

20. Walston–Felix Equation as Ratio Scale

Effort E = aS^b (where S is LOC).

Both effort and size are ratio-scale quantities; so their relationship preserves ratio scales.

---

21. Representation for Table 2.1

Assign numbers based on user preferences (e.g., more functionality gets higher number), only if survey shows clear ordering (60%+ of users).

---

22. Absolute Scale for "Number of Bugs"

Absolute measure: Count the number of bugs found in a test process.

"Number of bugs found" is not a measure of correctness—correctness is NOT the same as "no bugs found" (since bugs may still exist but undetected).

---

23. Application Domain Scale

a. Scale is **nominal** (categories).

b. Measure mode (most common domain among A-G):

A:5, B:1, C:1, D:1, E:2, F:6, G:5

Mode = 1 (WWW browser) and 5 (Operating systems).

c. Non-meaningful statement:

"System F is twice as application-domaind as System B"—nonsense as no numeric order exists.

---

24. Program Adaptability

a. Measure: Number of lines of code changed to add a feature.

Empirical relation: "Program X is easier to adapt than Program Y if fewer lines need to be changed."

b. Scale: Ratio (can be zero, ratios make sense).

c. Representation condition: If X needs fewer changes than Y in the real world, then M(X) < M(Y) numerically.

d. Pro: Objective, easy to count.

Con: May not account for effort required per line (some lines are harder to change).

Summary Cheat Sheet

- **Measurement** is about mapping real world to numbers: must preserve relationships (representation condition).
- **Scale types** determine what calculations and comparisons are meaningful:
  - Nominal: categories

    - Ordinal: order
    - Interval: order + equal gaps
    - Ratio: plus zero & ratios
    - Absolute: one unique way to count
- Only use math (mean, ratio) when supported by the scale.
- **Derived measurements** combine direct measures and must obey scale constraints.
